

Red ASM Security Final Report

SP-108 Red

Adam Llado | Kimani Gordon | Austin Abeln | Alec Sundby

CS4850 – Section 2 – Spring – 2026 – 05/04/2026 – Professor Sharon Perry

Website link:

https://red-asm-security.github.io/RedASMSecurity_Cyber_Capstone_Project/

Github Link:

https://github.com/Red-ASM-Security/RedASMSecurity_Cyber_Capstone_Project

STATS AND STATUS	
Lines of Code (LOC)	2,369
Components / Tools	• Slack Alerts & Cases Channels (Track & Triage Active Threats) • OWASP Juice Shop Cyber Security Training Website (Primary Vulnerable Service) • ASM Tools (Subfinder, DNSX, HTTPX, Nuclei, Ffuf) • Docker (Runs OWASP Juice Shop, Shuffle, TheHive) • Oracle Cloud Ampere Free Tier (4 CPUs, 24GB RAM) • Swapfile (MOST IMPORTANT – Expands Manager VM 12GB to 20GB) • Shuffle (Threat Response Automation Tool) • Wazuh Agent & Manager (Threat Detector & Alert Generator) • TheHive (Case Manager) • Nginx (Server to manage OWASP Juice Shop services) • ModSecurity (Firewall for OWASP Juice Shop) • Ubuntu 22.04 Minimal • MySQL Database • Grafana (Database Visualization) • GitHub (Version Control & Alerts Event Log) • Remmina Remote Desktop Client (App to Remote Desktop into Manager VM) • Cloudflare (Tunnel to internet to access private services running on Manager VM) • Buy a Domain (redasmsecurity.cloud)
Hours Estimate	400
Hours Actual	450
Status	Project is 100% complete

1.0 Project Overview / Abstract (Research)	3
2.0 Challenges, Solutions, and Assumptions	4
2.1 Challenges	4
2.2 Solutions	4
2.3 Assumptions	4
3.0 Requirements	5
3.1 Infrastructure Requirements	5
3.2 Software Requirements	5
3.3 ASM Tool Requirements	5
3.4 Security Stack Requirements	5
3.5 External Services	6
4.0 Analysis / Design	6
4.1 System Architecture	6
Target VM (Public)	6
Manager VM (Private)	6
4.2 Data Flow	6
5.0 Development	7
5.1 Development Documentation	7
5.2 Database Connection	7
5.3 Project Setup	8
Environment Setup	8
Target VM Setup	8
Manager VM Setup	8
6.0 Test Plan	9
6.1 Testing Scope	9
In Scope	9
Out of Scope	9
6.2 Test Objectives	9
7.0 Test Report	9
8.0 Version Control	10
8.1 GitHub Link	10
8.2 GitHub Folder Structure	11
9.0 Conclusion / Summary	11
10.0 Appendix	11
10.1 Shuffle Red ASM Security mini-Security Operations Center (SOC) Suite	11

10.2 Slack Threat Management Hub	13
10.3 Custom Wazuh SQL Injection Rules.....	13
10.4 Cloudflare Manager VM Internet Tunnel (Accessibility)	14
10.5 Grafana Database Visualization Tool.....	15
10.6 Runbooks for Threat Investigation Guidance	15
10.7 Attack Vulnerability Testing in frontend_paths_scan.py	16

1.0 Project Overview / Abstract (Research)

This project focuses on the implementation of an **Attack Surface Management (ASM) framework** to simulate both offensive and defensive cybersecurity operations. The system is designed to identify exposed assets within a domain and detect potential vulnerabilities that could be exploited by attackers.

The project demonstrates how ASM tools can be used to:

- Discover subdomains and services, mapping the attack surface
- Prevent cyber-attacks through ASM database collection and internal testing
- Identify vulnerabilities such as SQL injection
- Track changes in the attack surface over time
- Expose vulnerable services that attackers can exploit
- Prevent cyber-attacks through ASM database collection and internal testing

The system integrates ASM with a mini security operations center (SOC) consisting of:

- **Wazuh (SIEM)** for alert detection
- **Shuffle (SOAR)** for automated response
- **TheHive** for case management
- **Slack** for alert triage and investigation

The goal of the project is to showcase the duality of ASM, providing the necessary spark for both red and blue team to achieve their goals through attack surface reconnaissance; red team exploits vulnerabilities using SQL injection with blue team's simulated active threat response and cyber-attack prevention.

2.0 Challenges, Solutions, and Assumptions

2.1 Challenges

Limited System Resources

The manager VM was required to run multiple services including Wazuh, Shuffle, TheHive, MySQL, Grafana, and custom Python scripts on limited hardware (2 CPUs, 12GB RAM).

Shuffle Integration Complexity

Shuffle required strict workflow logic, including:

- Proper variable initialization
- Correct syntax for variable passing
- Data formatting between Python and Shuffle

System Stability Under Load

Running continuous scans and alert processing created performance bottlenecks and system instability.

2.2 Solutions

- Implemented an **8GB swapfile**, increasing usable memory from 12GB to 20GB
- Structured Shuffle workflows to ensure correct execution order
- Added Python preprocessing for data formatting and handling
- Optimized scan configurations to reduce system load

2.3 Assumptions

- Oracle Cloud Free Tier (ARM architecture) is used
- OWASP Juice Shop is the primary vulnerable application
- SQL injection is the primary attack vector
- Cloudflare is used for DNS and tunneling
- Slack is used for SOC-style alert triage
- redasmsecurity.cloud is used as the target scope

3.0 Requirements

3.1 Infrastructure Requirements

- Oracle Cloud (2 virtual machines)
 - Ubuntu 22.04 Minimal (ARM64)
 - Private and public subnet configuration
 - Ampere ARM free tier (4 CPUs, 24GB RAM)
 - SSH configuration
-

3.2 Software Requirements

- Docker (containerized services)
 - Nginx (reverse proxy)
 - MySQL / MariaDB (database)
 - Python 3 (automation scripts)
-

3.3 ASM Tool Requirements

- Subfinder (subdomain discovery)
 - DNSX (DNS resolution and brute force)
 - HTTPX (service probing)
 - Nuclei (vulnerability scanning)
 - FFUF (directory fuzzing)
-

3.4 Security Stack Requirements

- Wazuh (SIEM)
 - Shuffle (SOAR)
 - TheHive (case management)
 - ModSecurity (web application firewall)
-

3.5 External Services

- Cloudflare (DNS and tunneling)
- Slack (alert triage)
- GitHub (logging and version control)
- Buy a Domain

4.0 Analysis / Design

4.1 System Architecture

The system consists of two virtual machines:

Target VM (Public)

- Hosts OWASP Juice Shop (vulnerable and patched versions)
 - Uses Nginx as a reverse proxy
 - Protected by ModSecurity
 - Files to Process Slack Action Button blocks of juice shop path & IP addresses
 - Sends alert logs to the manager VM via Wazuh agent
-

Manager VM (Private)

- Hosts core security services:
 - Wazuh Manager and Dashboard
 - Shuffle automation platform
 - TheHive case management system
 - MySQL database
 - ASM scanning scripts
 - Grafana dashboards
 - Custom Python server listens for Slack Action Buttons
-

4.2 Data Flow

1. ASM tools discover subdomains and services
2. Data is processed and stored in the database
3. Vulnerabilities are detected using scanning tools
4. Attacker uses SQL injection on OWASP Juice Shop hosted service

5. Wazuh generates alerts
6. Shuffle processes alerts and determines severity
7. Alerts are sent to Slack and TheHive
8. Analysts respond using automated actions (block IP, contain path)

5.0 Development

5.1 Development Documentation

ASM Scanning Engine. The system is built using a modular Python architecture:

- `scanner.py` – main pipeline controller
- `terminal_run.py` – executes external tools
- `resolver.py` – processes HTTPX data & calls vulnerability functions
- `vulnerability_scan.py` – runs Nuclei
- `url_paths_scan.py` – performs directory fuzzing
- `db_handler.py` – manages database operations
- `frontend_paths_scan.py` – Legitimizes paths in Main.js & tests for cyber attack vulnerabilities

Key design decisions:

- Centralized command execution
- Modular pipeline structure
- File-based debugging
- Drift detection for asset tracking
- Internal cyber attack testing
- Independent column tracking for scan history

5.2 Database Connection

The project uses a MySQL database hosted on the manager VM.

Connection details:

- Host: 127.0.0.1 (via SSH tunnel)
- Port: 3306
- Database: `asm_database`

The database stores:

- Discovered assets
- Vulnerabilities
- URL paths
- Application endpoints
- Cyber Attack Vulnerabilities (SQLi, XSS, DoS)
- Patched status for cyber attacks

History tables track changes over time, allowing for analysis of security drift.

5.3 Project Setup

Environment Setup

- Two Oracle Cloud VMs created (Target and Manager)
- Ubuntu 22.04 Minimal installed on both
- SSH enabled for secure access
- Ampere ARM free tier (4 CPUs & 24GB RAM)

Target VM Setup

- Docker installed for application hosting
- Nginx configured for routing
- OWASP Juice Shop deployed (original & patched version)
- Wazuh agent installed
- Modsecurity enabled
- File added for Contain Path endpoint (Slack Action Button)

Manager VM Setup

- Wazuh manager installed
- Shuffle and TheHive deployed via Docker
- MySQL database configured
- ASM tools installed
- Grafana configured for visualization
- Crontab deployed for automated scanning
- Custom Wazuh rules for SQL injection
- Python Server to listen for Slack Action Buttons

6.0 Test Plan

The purpose of testing is to ensure the system functions correctly under both normal and attack conditions.

6.1 Testing Scope

In Scope

- Alert generation
- Automated response actions
- Database updates
- Slack notifications
- Case promotion logic

Out of Scope

- Authentication systems
 - External tool accuracy
 - Denial of service testing
-

6.2 Test Objectives

- Verify SQL injection detection
- Verify IP blocking functionality
- Ensure normal traffic is not blocked
- Validate alert-to-case promotion
- Confirm database updates

7.0 Test Report

All test cases were executed successfully.

Test Case	Result
Block IP	Pass
SQL Injection Detection	Pass
Normal Query Handling	Pass
Alert Promotion	Pass
Case Generation	Pass

The system successfully:

- Detected attacks
- Generated alerts
- Executed automated responses

8.0 Version Control

8.1 GitHub Link

Version control was managed using GitHub:

https://github.com/CS4850-ASMCyberProject/RedASMSecurity_Cyber_Capstone_Project

All source code, scripts, and documentation were maintained in the repository with updates committed throughout development.

8.2 GitHub Folder Structure

```
ASM_Manager/  
├─ ASM_Scanning_Engine/  
├─ Grafana/  
├─ MySQL_Database/  
├─ SOC_Suite/  
│   ├─ Shuffle/  
│   ├─ TheHive/  
│   └─ Wazuh_Manager/  
  
ASM_Target/  
├─ Dockerized_Juice_Shop_Services/  
├─ Modsecurity/  
├─ Nginx_Server/  
└─ Wazuh_Agent/  
  
Supporting_Services/  
├─ Cloudflare/  
└─ Slack/  
  
Website/
```



9.0 Conclusion / Summary

The Red ASM Security Project demonstrates a complete cybersecurity workflow that combines **attack surface discovery, vulnerability detection, automated response, & attack prevention**.

The system successfully integrates multiple tools into a unified platform capable of:

- Identifying exposed assets
- Detecting vulnerabilities
- Generating alerts
- Automating response actions

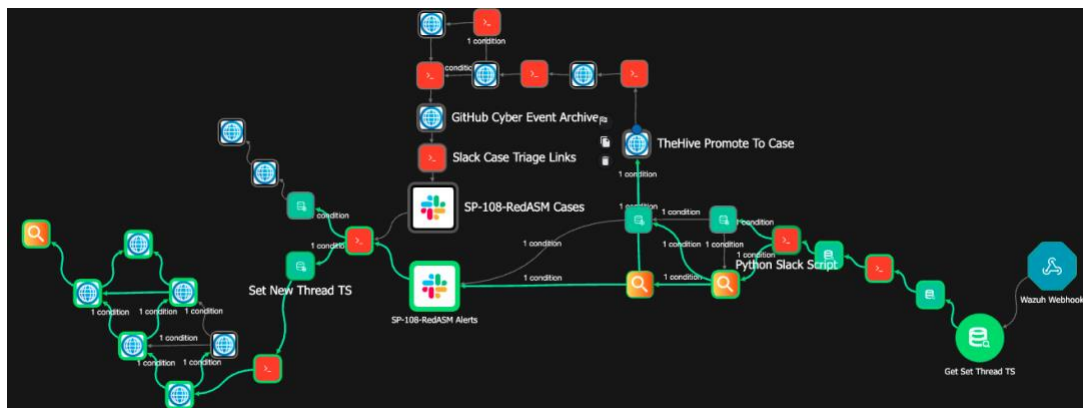
The project highlights the importance of using attacker-based methodologies to improve defensive security strategies. By continuously monitoring the attack surface and responding to threats in real time, the system provides a practical example of a modern SOC environment.

10.0 Appendix

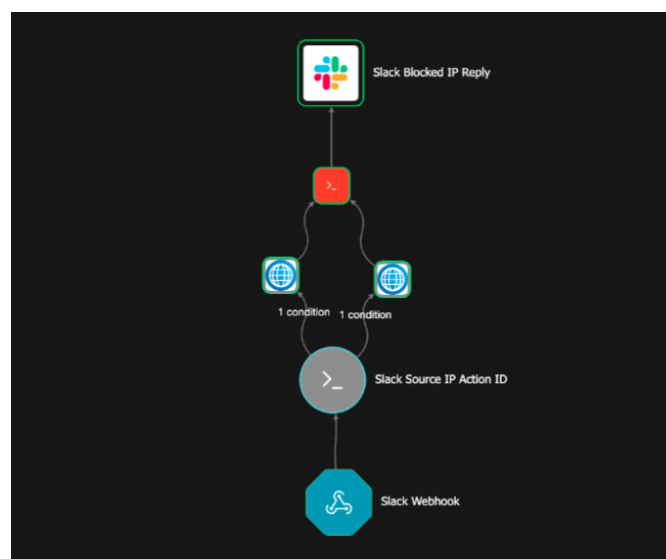
10.1 Shuffle Red ASM Security mini-Security Operations Center (SOC) Suite

By: Adam Llado

- 2 Workflows: Main SOC Engine & Slack Action Button Resolver
- Wazuh Webhook pulls data in
- Cache nodes for getting & setting information (Slack threading & similar alerts)
- Custom Python scripts to differentiate between high priority case & low priority alerts & building alert payloads
- Design & architecture built from scratch
- Logic to promote alerts to cases based on severity
- Slack Action Buttons & Triage Links added to cases
- Pages are added to TheHive for enriched investigation of internal python logic (case history, case scoring, data leaked)
- Wazuh sends alerts, Python generates alerts, Slack, TheHive, & Github are the alert endpoints



Shuffle SOC Workflow

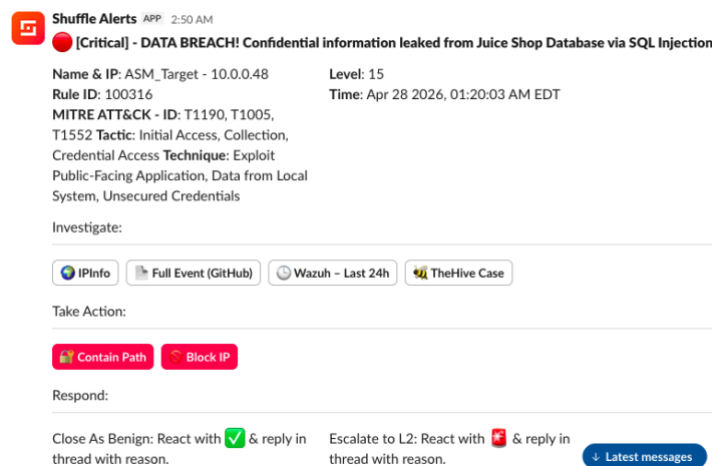


Shuffle Slack Action Button Resolver

10.2 Slack Threat Management Hub

By: Adam Llado

- Main endpoint for further investigation into active security threats
- 2 Channels: Alerts (low priority) & Cases (high priority)
- **Cases channel:**
 - IPInfo determines if IP is malicious
 - GitHub is an alert archive of original wazuh alerts
 - Wazuh shows activity for IP in the last 24 hours
 - TheHive provides detail into case history, case scoring, & data leaked
 - Slack Action Buttons: Contain Path (Secures Vulnerable Endpoint) & Block IP



Slack Cases Channel: Custom Alert Generated from Wazuh, Built in Shuffle, Sent to Slack

10.3 Custom Wazuh SQL Injection Rules

By: Adam Llado

- Alerts include additional SQL injection words and juice shop database table names for an enriched and targeted output
- **Alert 100313:** General custom SQL injection alert that listens for juice shop table names and reserved SQL language to generate tailored alert
- **Alert 100314:** If the term sqlite_master is used, an attempt to get the database scheme, this alert is generated
- **Alert 100314:** If the return for alert 100313 is successful, this alert is generated to inform analysts a data breach has occurred
- Alerts used in Video Presentation

```

<group name="juice_shop_custom_sql_injection,">
  <rule id="100313" level="12">
    <if_sid>31103</if_sid>
    <url type="pre2">(?!)(%20|\\+){addresses|basketitems|baskets|captchas|cards|challenges|complaints|deliveries|feedbacks|hints|imagecaptchas|memories|orders|passwords|payments|products|reviews|shipping|subscriptions|support|terms|tokens|transactions|users|vouchers|wishlist}</url>
    <description>WARNING! SQL Injection targeting Juice Shop database tables. The confidentiality of database information has potentially been breached.</description>
    <mitre>
      <id>T1190</id>
      <id>T1005</id>
      <id>T1552</id>
    </mitre>
    <group>attack,sql_injection,data_exposure,confidentiality_breach,critical</group>
  </rule>

  <rule id="100314" level="12">
    <if_sid>31103</if_sid>
    <url type="pre2">(?!)(%20|\\+){sqlite_master}</url>
    <description>WARNING! SQL Injection attempt using sqlite_master. The confidentiality of Juice Shop database schema potentially leaked.</description>
    <mitre>
      <id>T1190</id>
      <id>T1005</id>
      <id>T1552</id>
    </mitre>
    <group>attack,sql_injection,data_exposure,confidentiality_breach,critical</group>
  </rule>

  <rule id="100316" level="15">
    <if_sid>100313,100314</if_sid>
    <id>200|^304</id>
    <description>DATA BREACH! Confidential information leaked from Juice Shop Database via SQL Injection.</description>
    <mitre>
      <id>T1190</id>
      <id>T1005</id>
      <id>T1552</id>
    </mitre>
    <group>attack,sql_injection,data_exposure,confidentiality_breach,critical</group>
  </rule>
</group>

```

10.4 Cloudflare Manager VM Internet Tunnel (Accessibility)

By: Adam Llado

The Private Manager VM is inaccessible to the internet, but the integration of a Cloudflare tunnel allows the private services to be accessible to authorized users, making development & investigation convenient.

- Makes private services publicly available
- Access to:
 - Shuffle
 - Wazuh Manager Dashboard
 - TheHive Case Management
 - Grafana Database Visualization Tool

ASM_Manager

Overview CIDR routes Hostname routes **Beta** Published application routes Live logs

Published application routes

Allow your Tunnel to reach applications whose domains you connected to Cloudflare. [Manage connected domains](#)

+ Add a published application route

	Published application routes	Path	Service	Origin configurations	Menu
1	wazuh.redasmsecurity.cloud	*	https://10.0.0.97	1	⋮
2	slack_blockip.redasmsecurity.cloud	*	https://10.0.0.97:8443	1	⋮
3	shuffle.redasmsecurity.cloud	*	https://10.0.0.97:8443	1	⋮
4	thehive.redasmsecurity.cloud	*	https://10.0.0.97:3443	1	⋮
5	grafana.redasmsecurity.cloud	*	http://10.0.0.97:3000	1	⋮

10.5 Grafana Database Visualization Tool

By: Kimani Gordon

- Showcases database information in enriched & human readable format



10.6 Runbooks for Threat Investigation Guidance

By: Austin Abeln

- Runbooks for Brute Force, SQL Injection, & File Integrity attacks

Runbook: SQL Injection Detection (Web Application)

Applies to:

- Wazuh alerts related to web attacks
- Alerts from ModSecurity or custom detection rules
- Alert severity: High or Critical
- MITRE ATT&CK: T1190

Purpose

This runbook explains how to respond to SQL injection attempts detected on web applications.

Trigger Conditions

Use this runbook when:

- A Wazuh alert indicates possible SQL injection activity
- The alert contains suspicious SQL patterns or payloads
- The alert severity is high or critical

Common indicators:

- ' OR 1=1 --
- UNION SELECT
- SELECT * FROM
- sqlite_master
- Unexpected SQL errors in logs

Decision Guide

- If the request is blocked and no impact occurred → Log the event and monitor for repeated attempts
- If the request shows probing behavior (multiple payloads) → Flag as active attack and monitor closely
- If the request appears successful or returns data → Escalate immediately as potential data breach

Response Actions

- Block the source IP (firewall or WAF)
- Enable or tighten ModSecurity rules
- Validate and sanitize user inputs in the application
- Review database logs for unauthorized queries
- Patch vulnerable endpoints

10.7 Attack Vulnerability Testing in frontend_paths_scan.py

By: Alec Sundby

- After frontend paths are legitimized, Internal testing is done for SQLi, XSS, & DoS to discover vulnerabilities before they are exploited.

```
frontend_paths_scan.py

1 for path in confirmed_paths:
2     is_sql = any(words in path.lower() for words in sql_words)
3     is_xss = any(words in path.lower() for words in xss_words)
4     is_dos = any(words in path.lower() for words in dos_words)
5
6     data = {
7         'url_path': path,
8         'sql_candidate': is_sql,
9         'xss_candidate': is_xss,
10        'dos_candidate': is_dos,
11        'tested': False,
12        'vulnerable': False,
13        'vulnerable_to': []
14    }
15
16    if data['sql_candidate']:
17        data['tested'] = True
18        sqli = "apple%27%29%20UNION%20SELECT%20id%2Cid%2CfullName%2CcardNum%2CexpMonth%2CexpYear%2C7%2C8%2C9%20FROM%20cards%3B"
19        http_status, content_length = get_status(base_url, path, sqli, False, False)
20        if http_status in ["200", "301"] and content_length > 50:
21            data['vulnerable'] = True
22            data['vulnerable_to'].append("sql_injection")
23
24    if data['xss_candidate']:
25        data['tested'] = True
26        xss = "%3Cscript%3Ealert(1)%3C/script%3E"
27        response = get_status(base_url, path, xss, False, True)
28        if "<script>alert(1)</script>" in response:
29            data['vulnerable'] = True
30            data['vulnerable_to'].append("reflected_xss")
31
32    if data['dos_candidate']:
33        data['tested'] = True
34        codes = [get_status(base_url, path, "", True, False) for _ in range(2)]
35        if "429" not in codes:
36            data['vulnerable'] = True
37            data['vulnerable_to'].append("no_rate_limiting")
38
39    validated_payload.append(data)
40
41 return validated_payload
```

SQL Injection Testing

XSS Testing

Denial of Service Testing

